

餘燼啟示錄：開發建議

諮詢問題：現在是HarpoonTurret有問題

好的，資深遊戲開發架構師，我已仔細審閱了您提供的《餘燼航路：鏽蝕重生》相關數據和腳本。感謝小白龍與二姐的詳盡企劃與代碼片段，這為我們提供了非常寶貴的分析基礎。

從整體企劃層面來看，AI分析報告和新手教學設計都展示了團隊對核心玩法（橫向捲軸航行管理、類魂系Boss戰、塔防基地防禦）、資源系統、環境危險以及蒸氣動力船概念的深刻理解和高度契合。但在世界觀風格演進上，AI報告也精準指出了「生物龐克」與「魔法龐克」的偏離問題，並給出了「虛空異變的機械殘骸」與「古代高壓蒸氣科技的失控」的優雅改造方案，這非常有助於保持遊戲風格的一致性和獨特性。

現在，我們將焦點轉向 `**`HarpoonTurret.gd`**` 這個關鍵腳本。魚叉砲台是《餘燼航路：鏽蝕重生》中玩家與環境、敵人互動的核心機制之一，它橫跨了航行管理（捕獲資源）、戰鬥（拉扯怪物）和資源獲取（拆解怪物）等多個核心循環。從提供的腳本和相關聯的邏輯來看，``HarpoonTurret`` 確實存在一些 `**潛在的問題、邏輯衝突和未實現的依賴**`，這可能導致遊戲體驗不流暢甚至功能失效。

以下是我的詳細分析與建議：

`**《餘燼航路：鏽蝕重生》- `HarpoonTurret.gd` 腳本深度分析**`

`**總體評價：**`

``HarpoonTurret.gd`` 的設計理念良好，採用了狀態機管理砲台行為，並試圖將輸入、物理、輸出模組化。其對「連點R對抗」和「長按R收回」的設計非常符合「黃銅蒸氣」的硬核操作感，也契合了《餘燼航路》多工操作的設計初衷。然而，它與其他腳本的`**交互接口定義不夠明確、依賴的方法未在被依賴腳本中實現**`，以及`**物理狀態管理上的潛在衝突**`，是目前最突出的問題。

一、主要問題與潛在衝突點 (紅色警報)

1. **關鍵依賴方法未實現：`ShipManager` 蒸汽消耗機制缺失**

* **問題描述**：`HarpoonTurret` 在 `\_handle\_retraction\_logic` 和 `\_fire` 函數中調用了 `\_ship\_manager.can\_afford\_steam(cost)` 和 `\_ship\_manager.request\_steam\_consumption(cost)`。然而，從提供的 `ship\_manager.gd` 腳本片段來看，這些方法**並不存在**。

* **影響**：魚叉砲台的發射和收回（特別是拉扯怪物時的「連點R對抗」）將無法扣除蒸汽壓力，導致核心資源管理機制失效，玩家可以無限發射和拉扯。遊戲平衡將嚴重破壞。

* **優先級**：極高。

2. **`HarpoonProjectile` (魚叉子彈) 接口不完整**

* **問題描述**：

* `HarpoonTurret.\_handle\_retraction\_logic` 調用 `\_current\_harpoon.get\_attached\_target()`。此方法在 `harpoon.gd` 中**未定義**，儘管 `\_target\_body` 內部變量存在。

* `HarpoonTurret.\_fire` 中嘗試連接 `p.retracted\_to\_launcher` 信號，但 `harpoon.gd` **沒有定義**這個信號。`harpoon.gd` 內部通過 `launcher.complete\_retraction()` 回調砲台，但這種單向回調不夠靈活，且與 `HarpoonTurret` 中期望的信號模式不符。

* `HarpoonTurret.\_on\_target\_reached\_deck` 調用 `\_current\_harpoon.request\_retraction\_complete()`。此方法在 `harpoon.gd` 中**未定義**。

* **影響**：砲台無法得知魚叉勾住了什麼目標，無法啟動正確的收回邏輯，也無法在目標被拉上甲板或魚叉收回時正確地清理自身狀態和子彈實例。

3. **`MonsterBody` (怪物實體) 交互接口不完整**

* **問題描述**：

* `HarpoonTurret.\_handle\_retraction\_logic` 調用 `monster\_target.is\_alive()` (在 `monster.gd` 中已定義，但需要進一步確認其準確性，例如死亡後是否立即設為 `false`)、`monster\_target.get\_weight()` 和 `monster\_target.apply\_pull\_force(...)`。這兩個方法在 `monster.gd` 中**未定義**。

* `HarpoonTurret.\_on\_target\_reached\_deck` 調用

`target.get\_trapped()`。此方法在`monster.gd`中**未定義**。

* **影響：** 砲台無法判斷怪物是否存活、獲取其重量來計算拉力，也無法對怪物施加物理拉力，更無法通知怪物已成功被「捕獲上船」。這將導致拉扯怪物的核心玩法失效。

4. **`Player` (玩家角色) 砲台掛載/卸載機制不完整與物理衝突**

* **問題描述：**

* `HarpoonTurret` 的 `\_mount\_player` 和 `\_unmount\_player` 方法中，期望玩家腳本 (`player.gd`) 有 `mount\_to\_turret` 和 `\_unmount\_from\_turret` 方法。這些方法在 `player.gd` 中**未明確定義**其行為 (例如，玩家本身的 `\_physics\_process` 是否需要停用？)。

* **嚴重的物理衝突：** `HarpoonTurret.\_physics\_process` 中有 `\_operator.global\_position = global\_position`。這意味著砲台會**強制設置**玩家的全局位置。如果玩家自身的 `\_physics\_process` 仍在運行並試圖移動，將導致**嚴重的物理抖動和不穩定**。這是一種「上帝之手」式的直接操控，與 `CharacterBody2D` 的 `move\_and\_slide()` 邏輯相悖。

* **影響：** 玩家在操作砲台時可能會不斷抖動、被甩開，或者無法獲得預期的操作體驗。砲台操控模式的核心穩定性受到威脅。

5. **`Global.hooked\_monsters` 列表管理缺失**

* **問題描述：** `Ship.gd` 在計算怪物阻力時，會遍歷 `Global.hooked\_monsters` 列表。但從 `HarpoonTurret.gd` 中，**沒有看到**將怪物添加到此列表或從中移除的邏輯。

* **影響：** 船隻無法正確計算因勾住怪物而產生的額外阻力，導致航行和壓力消耗的平衡性問題。

6. **非 `MonsterBody` 目標處理不足**

* **問題描述：** `\_handle\_retraction\_logic` 主要處理 `target is MonsterBody` 的情況。如果魚叉勾住了 `StaticBody2D` (如 `cabinet.gd` 或 `obstacle.gd` 提到的「補給箱」)，則會進入 `pass` 語句或 `elif is\_instance\_valid(target): pass`，導致無法拉取這些非怪物的可互動物體。

* **影響：** 玩家無法使用魚叉拉取資源箱或其他環境物體，這將大幅限制魚叉砲台的通用性，與新手教學中「用魚叉勾住補給箱」的設計不符。

二、風格改造與實作思路 (解決方案與優化建議)

為了保持「黃銅蒸氣末世」的硬核與深度，同時解決上述功能問題，以下是具體的改造與實作思路：

****1. 強化 `ShipManager.gd` (核心) : ****

* **職責：** `ShipManager` 作為全局的船隻狀態與資源管理者，必須提供統一的接口來處理蒸汽的消耗與補充。

* **改造：** 在 `ship\_manager.gd` 中新增以下方法：

```
` ``gdscript
# ship_manager.gd (補充)
signal steam_pressure_changed(current_pressure: float,
max_pressure: float) # 供 UI 或其他組件監聽

func can_afford_steam(amount: float) -> bool:
    return steam_pressure >= amount

func request_steam_consumption(amount: float) -> void:
    if can_afford_steam(amount):
        steam_pressure -= amount
        steam_pressure = max(steam_pressure, 0.0) # 確保不為負
        steam_pressure_changed.emit(steam_pressure,
max_pressure)
        # 可以觸發視覺或音效提示，例如 Global.steam_strain.emit(amount)
    else:
        print("⚠ [ShipManager] 蒸汽不足，無法執行操作。")
        # 可以觸發「蒸汽不足」的UI提示
```

```
func add_steam_pressure(amount: float) -> void:
    steam_pressure += amount
    steam_pressure = min(steam_pressure, max_pressure) # 確保不
超過上限
    steam_pressure_changed.emit(steam_pressure, max_pressure)
...

```

* **影響：** `HarpoonTurret` 和其他所有需要消耗或補充蒸汽的組件，都將通過這些統一且安全的接口與 `ShipManager` 交互。

****2. 完善 `HarpoonProjectile.gd` (魚叉子彈) : ****

* **職責：** 魚叉子彈應能告知其掛鉤狀態、目標，並提供一個統一的收回信號。

* **改造：**

``gdscript

harpoon.gd (補充/修正)

signal retracted_to_launcher(harpoon_instance: HarpoonProjectile)

新增此信號

... (原有代碼) ...

新增：公開目標的方法

func get_attached_target() -> Node2D:

return _target_body

修正：start_pullback 應標記魚叉正在被拉回

func start_pullback():

_is_pulled = true

_is_active = false

_is_waiting_recovery = false

set_collision_layer_value(1, false) # 關閉自身碰撞

set_collision_mask_value(1, false)

新增：在收到拉回指令時，確保移除與目標的強制位置同步

if is_instance_valid(_target_body) and

_target_body.has_method("release_harpoon_lock"):

_target_body.release_harpoon_lock(self) # 告知目標解除魚叉的物理固定

修正：統一魚叉的生命週期結束方式

func request_retraction_complete():

確保只執行一次

if _is_pulled and

global_position.distance_to(launcher.global_position) < 50.0:

return # 已經在歸位途中或已歸位

if is_instance_valid(_target_body) and

_target_body.has_method("release_harpoon_lock"):

```
        _target_body.release_harpoon_lock(self) # 告知目標解除魚叉的  
物理固定
```

```
    # 開始物理回歸流程  
    start_pullback() # 觸發實際的物理回歸動畫  
  
    # 在歸位完成時觸發信號，而不是直接刪除  
    get_tree().create_timer(0.3).timeout.connect(func(): # 預留動畫時  
間  
        if is_instance_valid(self):  
            retracted_to_launcher.emit(self) # 歸位完成，發送信號  
            queue_free() # 刪除子彈實體  
    )
```

```
    # harpoon.gd 中 _on_impact 函數的修正  
func _on_impact(collision):  
    var hit_obj = collision.get_collider()  
    if hit_obj:  
        _target_body = hit_obj  
        _is_active = false  
        _is_waiting_recovery = false  
        _hit_offset = global_position - _target_body.global_position  
  
    # 新增：如果目標是可固定實體，通知其固定魚叉  
    if _target_body.has_method("attach_harpoon_lock"):  
        _target_body.attach_harpoon_lock(self, _hit_offset)  
  
    # 命中反饋：讓砲台知道現在可以開始「連點 R」了  
    if launcher:  
        launcher._current_harpoon = self # 這裡會被 HarpoonTurret
```

引用

```
launcher._set_turret_state(HarpoonTurret.TurretState.HARPOON_OUT)  
# 確保狀態同步
```

```
func _physics_process(delta: float) -> void:  
    # 狀態 A：回收歸位中
```

```

        if _is_pulled:
            var dir = (launcher.global_position -
global_position).normalized()
            var pull_speed = 2200.0 # 快速回歸
            global_position += dir * pull_speed * delta

            if global_position.distance_to(launcher.global_position) < 50.0:
                if is_instance_valid(launcher) and
launcher.has_method("complete_retraction"):
                    launcher.complete_retraction(self) # 讓砲台顯示備彈圖，並
清理 _current_harpoon
                    queue_free() # 刪除子彈實體
                    return
            # ... (其餘邏輯不變) ...
    ...

```

* **影響：** 魚叉和砲台之間的狀態同步將更加可靠，魚叉的生命週期管理將更為清晰。

****3. 完善 `MonsterBody.gd` (怪物實體)：****

* **職責：** 怪物應能響應魚叉的拉力，並在成功捕獲時切換狀態。

* **改造：** 在 `monster.gd` 中新增以下方法：

```

` ``gdscript
# monster.gd (補充)
@export var weight_class: float = 150.0 # 導出怪物重量，影響拉扯難度

# 新增：用於魚叉物理固定
var _attached_harpoons: Dictionary = {} # 存儲 {harpoon_instance:
hit_offset}

```

```

func get_weight() -> float:
    return weight_class

```

```

func apply_pull_force(force_vec: Vector2) -> void:
    if is_alive and not is_on_ship: # 只有活著且不在船上的才受拉力影響
        velocity += force_vec # 直接疊加，讓怪物有被拉扯的慣性

```

```

func get_trapped() -> void:

```

```

if is_alive: # 只有活著的怪物才能被「捕獲」
    is_on_ship = true
    is_hooked = false
    velocity = Vector2.ZERO # 停止移動
    print("⚓ [系統] 怪物成功被捕獲上甲板！")
# 通知 Global 移除該怪物從 hooked_monsters 列表，因為它已經上甲板
了

    if Global.hooked_monsters.has(self):
        Global.hooked_monsters.erase(self)

# 鬆開所有魚叉鎖定
    for harpoon_inst in _attached_harpoons.keys():
        if is_instance_valid(harpoon_inst) and
harpoon_inst.has_method("request_retraction_complete"):
            harpoon_inst.request_retraction_complete() # 魚叉自行收
回

        _attached_harpoons.clear()

# 改變行為：準備被拆解
    prepare_for_dismantle() # 假設有此方法，改變視覺或行為

func prepare_for_dismantle():
    # 怪物視覺變為「已死亡」或「暈眩」狀態，並標記可拆解
    is_alive = false
    set("is_ready_for_dismantle", true) # Player.gd 會檢查這個屬性
    # 可以播放粒子效果、改變貼圖、禁用碰撞體等
    print("💣 [怪物] 準備被拆解。")

# 新增：魚叉固定/解除鎖定接口 (配合 HarpoonProjectile)
    func attach_harpoon_lock(harpoon_inst: HarpoonProjectile, offset:
Vector2):
        _attached_harpoons[harpoon_inst] = offset
        # 在這裡可以讓怪物播放掙扎動畫、減少移動速度等
        is_hooked = true
        # 添加到 Global 的掛鉤列表
        if not Global.hooked_monsters.has(self):
            Global.hooked_monsters.append(self)

```



```
print("🦂 [怪物] 已被魚叉勾住並添加到全局列表。")
```

```
func release_harpoon_lock(harpoon_inst: HarpoonProjectile):  
    if _attached_harpoons.has(harpoon_inst):  
        _attached_harpoons.erase(harpoon_inst)  
        if _attached_harpoons.is_empty(): # 所有魚叉都鬆開了  
            is_hooked = false  
        # 從 Global 的掛鉤列表移除  
        if Global.hooked_monsters.has(self):  
            Global.hooked_monsters.erase(self)  
        print("🦂 [怪物] 已從全局列表移除。")  
    ...
```

* **影響:** 怪物將正確響應魚叉拉力，船隻能正確計算阻力，並且捕獲上船的流程將完整。

****4. 修正 `Player.gd` (玩家角色) 砲台交互與物理管理:****

* **職責:** 玩家應在其自身腳本中管理「是否由其他節點控制其物理」的狀態。

* **改造:**

``gdscript

player.gd (補充/修正)

var _mounted_turret_ref: Node2D = null # 儲存當前掛載的砲台引用

```
func mount_to_turret(turret_node: Node2D) -> void:
```

```
    _mounted_turret_ref = turret_node
```

```
    _is_on_turret = true
```

```
    set_physics_process(false) # 關鍵：停止玩家自身的物理處理
```

```
    global_position = turret_node.global_position + Vector2(0, 50) #
```

可以是砲台預設的玩家操作位置

```
# 視覺：可以隱藏玩家自身或切換到操作動畫
```

```
# get_node("Sprite2D").visible = false
```

```
func unmount_from_turret(target_pos: Vector2) -> void:
```

```
    _mounted_turret_ref = null
```

```
    _is_on_turret = false
```

```
    set_physics_process(true) # 關鍵：恢復玩家自身的物理處理
```

```

    global_position = target_pos # 設置玩家離開砲台後的位置

# 視覺：恢復玩家自身可見性或動畫
    # get_node("Sprite2D").visible = true

# 移除 HarpoonTurret 中 player_ref.global_position 的直接設置
# 相反，在 HarpoonTurret._physics_process 中，僅在 is_operating 狀態
下，
    # 如果 _operator 實例有效，則可以更新其動畫或傳遞視覺數據，但不要直接改
其 global_position。
    # 玩家的物理處理已由 set_physics_process(false) 停止，其位置由
mount_to_turret 設定一次即可。
    ...

* **影響：** 玩家在操作砲台時將不再與自身的物理引擎衝突，體驗將變得穩定
流暢。

**5. 強化 `HarpoonTurret.gd` 自身邏輯與非怪物目標處理：**
* **職責：** 砲台應處理所有可勾住的目標類型，並在需要時將其拉回。
* **改造：**
    ``gdscript
    # harpoon_turret.gd (修正/補充)
    # ... (原有代碼) ...

# 修正 _fire 函數中連接信號的方式
func _fire(direction: Vector2) -> void:
    # ... (原有檢查) ...

    _set_turret_state(TurretState.FIRING)
    _ship_manager.request_steam_consumption(fire_cost)

    var p = projectile_scene.instantiate() as HarpoonProjectile # 強類
型，使用 HarpoonProjectile
    _current_harpoon = p
    p.launcher = self

# 連接魚叉的收回信號，而不是期望它調用 complete_retraction
    if p.has_signal("retracted_to_launcher"):

```

```

        p.retracted_to_launcher.connect(complete_retraction)

    get_tree().current_scene.add_child(p)
    p.global_position = global_position

    p.launch(direction, self)
    harpoon_fired.emit(p)

# ... (後座力動畫) ...
# 動畫結束後進入 HARPOON_OUT 狀態，這會在魚叉成功發射後由 tween
回調

# complete_retraction 函數修正，現在它由魚叉的信號觸發
func complete_retraction(retracted_harpoon: HarpoonProjectile):
    if _current_harpoon != retracted_harpoon:
        # 如果這個信號不是來自我們當前發射的魚叉，則忽略
        return

    _current_harpoon = null
    _set_turret_state(TurretState.RELOADING) # 進入裝填狀態
    harpoon_reloaded.emit()

# _handle_retraction_logic 函數修正：處理不同目標類型
func _handle_retraction_logic(intent: Dictionary, delta: float) ->
void:
    if not is_instance_valid(_current_harpoon) or not
is_instance_valid(_ship_manager):
        _set_turret_state(TurretState.RELOADING)
        _current_harpoon = null
        return

    var target = _current_harpoon.get_attached_target() # 現在這個
方法應該存在於 HarpoonProjectile

    var pull_force = 0.0
    var steam_cost = 0.0

```

```

if target is MonsterBody: # 勾住了怪物
    var monster_target: MonsterBody = target
    var is_struggling = monster_target.is_alive

    if is_struggling:
        if intent.collect_just_pressed: # 連點 R 對抗
            pull_force = PULL_FORCE_STRUGGLE
            steam_cost = STEAM_COST_STRUGGLE
            _apply_vibration(0.1)
            _set_turret_state(TurretState.RETRACTING_STRUGGLE)
        elif _current_state == TurretState.RETRACTING_STRUGGLE
and not intent.collect_pressing:
            _set_turret_state(TurretState.HARPOON_OUT) # 中斷連點
            後回到 HARPOON_OUT
        else: # 目標不再掙扎 (已死亡或被擊暈)
            if intent.collect_pressing: # 長按 R 穩穩收回
                pull_force = PULL_FORCE_RETRIEVE
                steam_cost = STEAM_COST_RETRIEVE_PER_SEC * delta
                _set_turret_state(TurretState.RETRACTING_IDLE)
            elif _current_state != TurretState.HARPOON_OUT:
                _set_turret_state(TurretState.HARPOON_OUT)

    if pull_force > 0 and
_ship_manager.can_afford_steam(steam_cost):
        _ship_manager.request_steam_consumption(steam_cost)
        var pull_dir = (global_position -
monster_target.global_position).normalized()
        var mass = monster_target.get_weight() # 現在這個方法應該
存在於 MonsterBody
        monster_target.apply_pull_force(pull_dir * (pull_force / (1.0
+ mass/100.0)) * delta)

        if
global_position.distance_to(monster_target.global_position) <
DECK_REACH_DISTANCE:
            _on_target_reached_deck(monster_target)

```

```

        elif target is StaticBody2D or target is RigidBody2D: # 勾住了其他
            可拉取的物體 (如補給箱、浮木等)
            # 我們假設這些物體重量固定或不需要連點
            if intent.collect_pressing:
                pull_force = PULL_FORCE_RETRIEVE * 0.5 # 較小的拉力
                steam_cost = STEAM_COST_RETRIEVE_PER_SEC * 0.5 *
delta
                _set_turret_state(TurretState.RETRACTING_IDLE)
            elif _current_state != TurretState.HARPOON_OUT:
                _set_turret_state(TurretState.HARPOON_OUT)

            if pull_force > 0 and
            _ship_manager.can_afford_steam(steam_cost):
                _ship_manager.request_steam_consumption(steam_cost)
                var pull_dir = (global_position -
target.global_position).normalized()
                target.global_position += pull_dir * pull_force * delta # 直接
移動目標

                if global_position.distance_to(target.global_position) <
DECK_REACH_DISTANCE:
                    _on_collectable_reached_deck(target) # 新增處理函數
                else: # 未勾住任何目標，或者目標是不可拉取的
                    if _current_state != TurretState.HARPOON_OUT: # 只有在開始嘗
試收線後才考慮自動回收
                        _current_harpoon.request_retraction_complete() # 魚叉自行
收回

# 新增處理函數：處理非怪物可收集物到達甲板
func _on_collectable_reached_deck(target: Node2D):
    print("⚓ [系統] 可收集物體已到達甲板：", target.name)
    # 在這裡可以觸發物體的「上船」邏輯，例如添加到船上的貨艙
    # target.call("on_collected_by_ship", self) # 假設可收集物有此方法

# 清理魚叉
if is_instance_valid(_current_harpoon):
    _current_harpoon.request_retraction_complete()

```

```

# 移除 _operator.global_position = global_position
# 相關邏輯應由 Player.gd 的 mount_to_turret 處理
func _physics_process(delta: float) -> void:
    if not is_instance_valid(_operator):
        set_physics_process(false)
        return
    set_physics_process(true)

# 移除這行，由 Player 自身管理其位置，但確保其物理處理已禁用
# _operator.global_position = global_position

# ... (其餘邏輯不變) ...

```

* **影響：** 砲台的邏輯將更為健壯，能夠處理多種目標類型，並確保與其他系統的交互正確。

總結與實施路線圖：

1. **優先級最高 (橙色警報):**

- * **實現 `ShipManager` 缺失方法：** `can\_afford\_steam` 和 `request\_steam\_consumption`。
- * **實現 `MonsterBody` 缺失方法：** `get\_weight`、`apply\_pull\_force` 和 `get\_trapped`。
- * **修正 `HarpoonProjectile` 接口：** `get\_attached\_target` 方法，並添加 `retracted\_to\_launcher` 信號，以及 `request\_retraction\_complete` 邏輯。
- * **修正 `Player` 物理控制：** 在 `player.gd` 中實現 `mount\_to\_turret` 和 `unmount\_from\_turret`，確保在掛載時徹底禁用玩家的物理處理。

2. **次高優先級 (黃色警報):**

- * **移除 `HarpoonTurret` 的直接玩家位置鎖定：** 從 `HarpoonTurret.\_physics\_process` 中移除 `\_operator.global\_position = global\_position`。
- * **完善 `HarpoonTurret` 的 `\_handle\_retraction\_logic`：** 增加對

`StaticBody2D` (如補給箱) 的處理邏輯。

- * **實現`Global.hooked\_monsters`管理:** 在`MonsterBody`的`attach\_harpoon\_lock`中加入列表，在`release\_harpoon\_lock`或`get\_trapped`中移除。

3. **優化與細節 (藍色建議):**

- * 為`MonsterBody`的`is\_alive`屬性設置`setter`，確保死亡時能觸發相應的邏輯（例如停止移動、粒子效果等）。

- * 考慮為`HarpoonTurret`的非怪物目標拉回添加視覺或音效反饋。

- * 確保`cabinet.gd` (或其他可拉取資源箱) 有一個`on\_collected\_by\_ship`類似的方法來響應被拉上船。

- * 在`player.gd`中，當玩家手持物品時，應限制其無法操作砲台（此邏輯已在`HarpoonTurret.\_mount\_player`中檢查`\_nearby\_player.is\_carrying\_item()`，確保其可靠）。

透過以上細緻的分析與改造，我們將能解決`HarpoonTurret`當前存在的深層問題，使其成為《餘燼航路：鏽蝕重生》中一個功能健全、體驗流暢且富有蒸汽龐克特色的核心玩法組件。這將極大地提升遊戲的穩定性與可玩性。